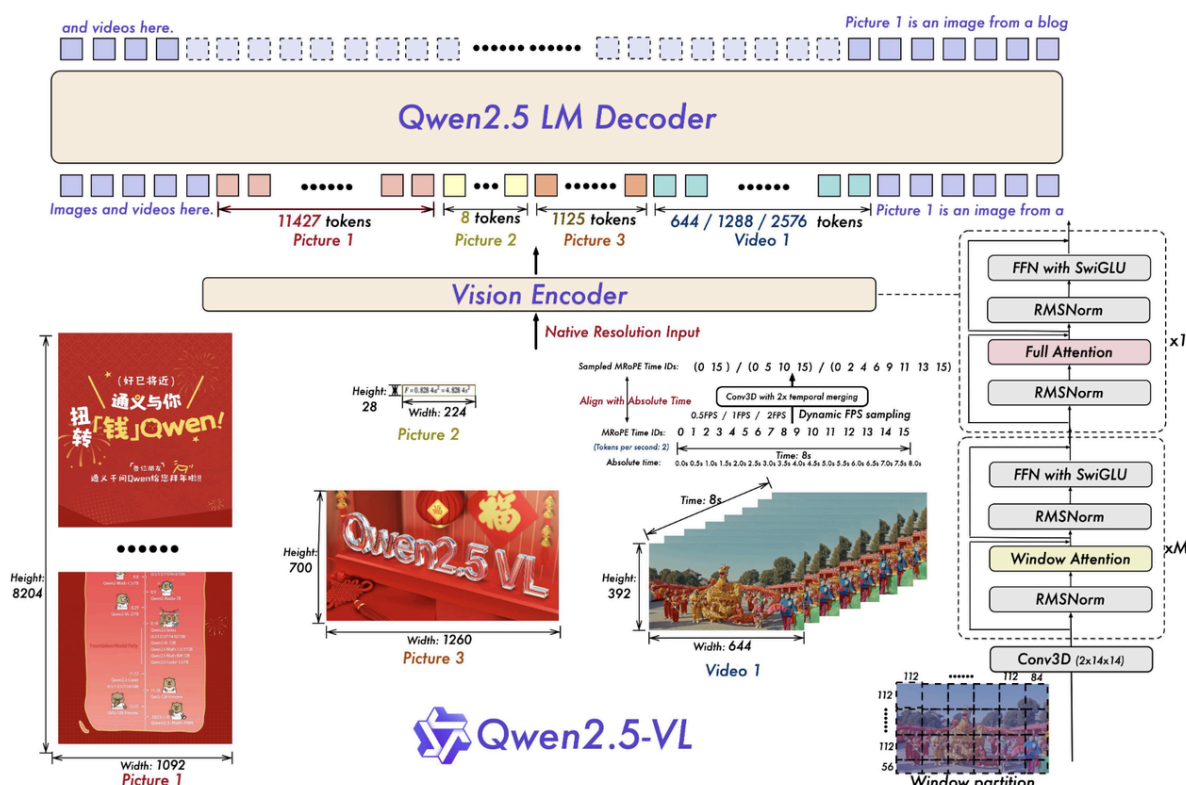


之前的帖子 [Qwen2.5-VL](#) 主要从transformers的PR对新模型的特性进行推测，现在随着官方 [Qwen2.5-VL Technical Report](#) 的发布，这期将更详细的解析Owen2.5-VL

1. 模型架构

从图就可以看出，本质上更新最大的模块还是在ViT视觉模块上：1) 用了Interleaved Window Attention 2) 使用了类似LLM的FFN和Norm；除了ViT这一点，在LLM上的改进是MRoPE对齐了绝对时间（Absolute Time）。



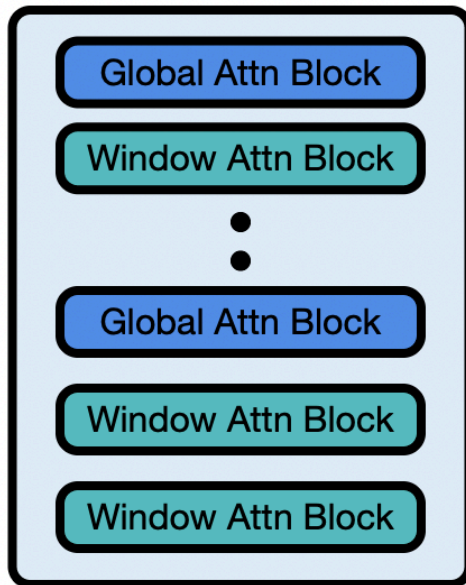
1.1 ViT 改进 (重点)

1.1.1 Window Attention

原有的Attention计算的关键问题是 $O(n^2)$ 的计算复杂度，尤其对于视觉ViT的长Visual Token序列。因此Qwen2.5-VL的ViT使用Window Attention，即attention的计算只在window内进行，在window之间是彼此独立的。

实际操作中将Global Attention和Window Attention**交错配置**：即只有4层使用Self Attention，其余层配置Window Attention

Qwen-2.5



```
for layer_num, blk in enumerate(self.blocks):
    if layer_num in self.fullatt_block_indexes:
        cu_seqlens_now = cu_seqlenselse:
            cu_seqlens_now = cu_window_seqlensif self.gradient_checkpoint
ing and self.training:
            hidden_states = self._gradient_checkpointing_func(
                blk.__call__, hidden_states, cu_seqlens_now, rotary_pos_e
mb)
    else:
        hidden_states = blk(
            hidden_states,
            cu_seqlens=cu_seqlens_now,
            rotary_pos_emb=rotary_pos_emb,)
```

其中，设置7，15，23，31层为全局attention，其余为window attention：

```
fullatt_block_indexes=list(range(7, 32, 8)),
```

其中获取cu_seqlens_now (window attention分割列表) 的方法参考 `get_window_index(self, grid_thw)`

1.1.2 MLP

ViT的MLP做出了更像语言模型架构的策略：

- 使用SwiGLU的FFN结构， `hidden_act="silu"` (参考 [1.1.5 前馈网络 FFN 和 激活函数 Activation](#))
- 标准化使用 RMSNorm。
- 其中的线性层增加bias，注意这里在llama这样的语言模型中是默认无偏置。

```
class Qwen2_5_VLMLP(nn.Module):
    def __init__(self, config, bias: bool = False):
        super().__init__()
        self.hidden_size = config.hidden_size
        self.intermediate_size = config.intermediate_size
        self.gate_proj = nn.Linear(self.hidden_size, self.intermediate_size, bias=bias)
        self.up_proj = nn.Linear(self.hidden_size, self.intermediate_size, bias=bias)
        self.down_proj = nn.Linear(self.intermediate_size, self.hidden_size, bias=bias)
        self.act_fn = ACT2FN[config.hidden_act]

    def forward(self, hidden_state):
        return self.down_proj(
            self.act_fn(self.gate_proj(hidden_state)) * self.up_proj(hidden_state)
        )
```

这里也对比给出 [Qwen2-VL](#) 的VisionMLP：

```
class VisionMlp(nn.Module):
    def __init__(self, dim: int, hidden_dim: int, hidden_act: str) -> None:
        super().__init__()
        self.fc1 = nn.Linear(dim, hidden_dim)
        self.act = ACT2FN[hidden_act]
        self.fc2 = nn.Linear(hidden_dim, dim)
```

```
def forward(self, x) -> torch.Tensor:
    return self.fc2(self.act(self.fc1(x)))
```

这里的激活函数用的是gelu，即 `hidden_act="quick_gelu"`

1.1.3 2D-RoPE

注意这里使用的仍然是和[Qwen2-VL](#)一模一样的2D-RoPE：

```
head_dim = config.embed_dim // config.num_heads
self.rotary_pos_emb = VisionRotaryEmbedding(head_dim // 2)
```

将hidden_dim的维度均分为x轴的RoPE和y轴的RoPE，以表达2D信息，详细解析参考[Qwen2-VL](#)

1.1.4 ViT的预训练

Qwen2.5-VL从头开始预训练ViT：包括三阶段CLIP pre-training、vision-language alignment、end-to-end fine-tuning，在训练中会进行**分辨率的动态抽样**。

1.2 PatchMerger

qwen2.5-VL的PatchMerger模块相比于qwen2-VL仅仅只是修改了LayerNorm为RMSNorm：

```
class Qwen2_5_VLPatchMerger(PatchMerger):
    def __init__(self, dim: int, context_dim: int, spatial_merge_size: int = 2) -> None:
        super().__init__(dim, context_dim, spatial_merge_size)
        self.ln_q = Qwen2RMSNorm(context_dim, eps=1e-6)
```

回顾一下qwen2-VL的PatchMerger模块，这里参考[Qwen2-VL](#)，为了控制seq_len，在MLP之前把相邻的2*2个vision token压缩成一个(其实就是在hidden_dim上拼接)，之后再过双层MLP (PatchMerger)：

```
class PatchMerger(nn.Module):
    def __init__(self, dim: int, context_dim: int, spatial_merge_size: int = 2) -> None:
        super().__init__()
        self.hidden_size = context_dim * (spatial_merge_size**2)
        self.ln_q = LayerNorm(context_dim, eps=1e-6)
        self.mlp = nn.Sequential(
            nn.Linear(self.hidden_size, self.hidden_size),
```

```

        nn.GELU(),
        nn.Linear(self.hidden_size, dim),
    )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.mlp(self.ln_q(x).view(-1, self.hidden_size))
        return x

```

1.3 LLM

语言模型基座用的是 Qwen2.5 LLM，主要的结构修改就是使用了对齐绝对时间 (Aligned to Absolute Time) 的MRoPE。

1.3.1 Time-Absolute MRoPE

MRoPE参考 [Qwen2-VL](#)，我们来回顾下naive MRoPE是怎么做的：

对于文本输入，所有三个维度都使用相同的位置ID，即等价于1D-RoPE。

对于视觉输入（图像和视频）：

Temporal (时间维度)：

- 对于图像，由于只有一帧，时间ID保持不变。
- 对于视频，每一帧的时间ID会递增，这意味着视频中的每一帧会有不同的时间ID，用于区分不同的时间点。

Height (高度维度)：

- 对于图像和视频，**高度ID**基于每个token在当前帧的空间位置进行唯一分配。这意味着每个视觉token的高度位置会有一个对应的ID，用于表示它在图像或视频中的纵向位置。

Width (宽度维度)：

- 对于图像和视频，**宽度ID**基于每个token在当前帧的空间位置进行唯一分配。这表示每个视觉token的宽度位置会有一个唯一的ID，用于表示它在图像或视频中的横向位置。

在 [Qwen2-VL](#) 基础上，Qwen2.5-VL支持带帧率的MRoPE，也就是说，只要提供了 **frame采样频率**，MRoPE中Temporal维度的编码是和现实世界的时间流动是对齐的，即可以换算成second秒。

Qwen2.5-VL主要变动就是在temporal position ids的编码依赖于公式：

$$\text{tokens_per_second} * \text{temporal_patch_size} / \text{fps}$$

其中 `tokens_per_second` 是指每秒被分成多少个时间点，`temporal_patch_size` 是指每一个时间点含有多少帧（因为为了防止帧数过多，需要合并相邻帧），`fps` 是帧率（帧数/秒）。

参考源码中 `Qwen2VLForConditionalGeneration.get_rope_index`，一看就可以知道Time-Absolute MRoPE的实现原理：

Temporal (Time): 3 patches, representing different segments of the video in time.

Height: 2 patches, dividing each frame vertically.

Width: 2 patches, dividing each frame horizontally.

We also have some important parameters:

fps (Frames Per Second): The video's frame rate, set to 1. This means one frame is processed each second.

tokens per second: This is a crucial parameter. It dictates how many "time-steps" or "temporal tokens" are conceptually packed into a one-second interval of the video. In this case, we have 25 tokens per second. So each second of the video will be represented with 25 separate time points. It essentially defines the temporal granularity.

temporal_patch_size: The number of frames that compose one temporal patch. Here, it's 2 frames.

interval: The step size for the temporal position IDs, calculated as $\text{tokens_per_second} * \text{temporal_patch_size} / \text{fps}$. In this case, $25 * 2 / 1 = 50$. This means that each temporal patch will have a difference of 50 in the temporal position IDs.

input_ids: [V V V V V V V V V V V V T T T T T], here V is for vision.
vision temporal position_ids: [0, 0, 0, 0, 50, 50, 50, 50, 100, 100, 100, 100]

vision height position ids: [0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1]

vision width position_ids: [0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1]

text temporal position_ids: [101, 102, 103, 104, 105]

text height position_ids: [101, 102, 103, 104, 105]

text width position_ids: [101, 102, 103, 104, 105]

Here we calculate the text start position_ids as the max vision position_ids plus 1.

具体方法的实现参考：

```
def get_rope_index(
    self,
    input_ids: torch.LongTensor,
    image_grid_thw: Optional[torch.LongTensor] = None,
    video_grid_thw: Optional[torch.LongTensor] = None,
    second_per_grid_ts: Optional[torch.Tensor] = None,
    attention_mask: Optional[torch.Tensor] = None,
)
```

2. 预训练

2.1 数据构成

相比Qwen2-VL，Qwen2.5-VL的预训练数据集从1.2T tokens扩展到约4T tokens，涵盖了多种多模态数据。数据集的主要组成部分包括：

数据类型	构建方法
Image captions	这类数据用于图像和文本的配对，以帮助模型理解图像内容与语言之间的关系。数据来源包括公开的图像数据集、社交媒体、以及具有描述性文本的图像。
Interleaved image-text data	为了提高这类数据的质量，采用了数据清洗和评分系统，尤其注重 文本-图像相关性 、 信息补充性 和 信息密度平衡 等方面。
Optical character recognition (OCR) data	构建了一个 多语言OCR数据集 ，涵盖了法语、德语、意大利语、西班牙语、葡萄牙语、阿拉伯语、俄语、日语、韩语和越南语等多种语言。
Visual knowledge data (e.g., celebrity identification, landmarks, flora, and fauna)	包括 名人、地标、动植物等的识别数据 ，帮助模型理解常见物体、地点、和生物的视觉表现。
Multi-modal academic questions	这类数据主要由多模态问答（VQA）数据集构成，涵盖了视觉与文本结合的复杂问题。
Localization data	定位数据帮助模型理解图像中不同对象的位置和空间关系。通过这种数据，模型可以进行物体定位、区域识别和空间推理。
Document parsing data	文档解析数据的构建特别注重表格、图表、公式和文本内容的处理。 Qwen2.5-VL通过创新性的方法将这些文档元素的布局信息和描述整合进HTML标签结构中。
Video descriptions and video localization	视频数据的构建考虑到了不同帧率（FPS）和不同时长的视频输入，采用动态采样的方法以平衡不同帧率的视频数据。

数据类型	构建方法
Agent-based interaction data	为了提升Qwen2.5-VL的代理能力，收集了移动平台、Web平台和桌面平台的截图，并生成了相关的描述与UI元素的定位注释。

2.2 三阶段 Pretrain

Stages	Visual Pre-Training	Multimodal Pre-Training	Long-Context Pre-Training
Data	Image Caption Knowledge OCR	+ Pure text Interleaved Data VQA, Video Grounding, Agent	+ Long Video Long Agent Long Document
Tokens	1.5T	2T	0.6T
Sequence length	8192	8192	32768
Training	ViT	ViT & LLM	ViT & LLM

Table 2: Training data volume and composition across different stages.

- 权重初始化：
LLM 的权重直接初始化于 Qwen2.5 LLM，ViT使用 DataComp 从0进行训练。
- Stage 1:
只训练ViT部分（论文说ViT，但是我感觉是不是PatchMerger也一起训练了），数据包括 Image Caption、Knowledge、OCR，使视觉部分与文本数据进行对齐。
- Stage 2:
整个模型（包括ViT和LLM）一起进行训练，使用了更多样的多模态数据，如视频数据、视觉问答等，以提升模型的多模态处理能力。
- Stage 3:
(没说冻结哪些参数，猜测是全部可训练) 第三阶段通过增加序列长度和加入视频与代理数据，使模型能够处理更长的上下文和更复杂的推理任务。

3. 后训练

在Qwen2.5-VL的后训练（微调）阶段，模型采用了**监督微调（SFT）**和**直接偏好优化（DPO）**这两种方法来进一步优化其在特定任务中的表现，**均保持ViT冻结**。以下是详细的步骤和数据构建细节：

- 监督微调（SFT）阶段使用了结构化的指令数据，使模型能够有效学习如何根据指令执行任务。数据集包括了图像-文本数据、视频-文本数据等，以帮助模型适应不同的任务类型。

- 直接偏好优化（DPO）阶段通过人类偏好对模型进行细致优化，尤其在复杂的推理任务中，帮助模型提升推理精度。

3.1 SFT

3.1.1 数据构建

SFT阶段使用了大量的Instruction数据，构建的格式是ChatML format，Qwen2.5-VL认为构建对话样式的Instruction的优势是：

- **显式的对话角色标签（Explicit dialogue role tagging）**：这是指在多模态对话中，为每个发言添加明确的角色标签（例如用户或系统），以便区分不同角色的发言。这样可以帮助模型理解在多轮对话中的参与者角色，从而更准确地进行对话推理和生成。
- **结构化地注入视觉嵌入与文本指令（Structured injection of visual embeddings alongside textual instructions）**：这一点说明了在训练过程中，不仅提供文本指令，还将视觉嵌入（如图像或视频的特征）与文本指令一起结构化地注入到模型中。这使得模型在处理多模态输入时，能够同时理解和融合视觉和文本信息，从而提升多模态理解和生成的能力。
- **通过格式感知的打包方式（Format-aware packing）来保持跨模态位置关系（Preservation of cross-modal positional relationships）**：这意味着在训练数据的打包过程中，模型会保持图像、文本和其他模态的空间位置关系。这有助于在处理图像和文本的结合时，确保模型能够理解它们在空间上的对应关系（例如，图像中的某个物体与描述该物体的文本之间的关系）。

数据过滤：

- Stage 1: Domain-Specific Categorization，训练一个Qwen2-VL-Instag用于questionanswer (QA) pairs的**层次化分类**，会分8个主类&30个子类。
- Stage 2: Domain-Tailored Filtering，使用rule-based & model-base方法进行过滤：
 - rule-based：删除比如incomplete, truncated, or improperly formatted responses。
 - model-base：训练Qwen2.5VL series的Reward Model。

Reasoning 数据的 Rejection Sampling：主要的方法还是使用RM进行过滤。

3.1.2 训练

ViT冻结，the model is fine-tuned on diverse multimodal data, including image-text pairs, video, and pure text, sourced from general VQA, Rejection Sampling, and specialized datasets such as Document and OCR, Grounding, Video, and Agent-related tasks.

3.2 DPO

DPO讲的比较少：ViT仍然冻结，the DPO phase focuses exclusively on image-text and pure text data, utilizing preference data to align the model with human preferences, with each sample processed only once to ensure efficient optimization. This streamlined process enhances the model's cross-modal reasoning and task-specific performance while maintaining alignment with user intent.

4. 实验

视觉任务Benchmark：

Table 3: Performance of Qwen2.5-VL and State-of-the-art.

Datasets	Previous Open-source SoTA	Claude-3.5 Sonnet-0620	GPT-4o 0513	InternVL2.5 78B	Qwen2-VL 72B	Qwen2.5-VL 72B	Qwen2.5-VL 7B	Qwen2.5-VL 3B
College-level Problems								
MMMU _{val} (Yue et al., 2023)	70.1 Chen et al. (2024d)	68.3	69.1	70.1	64.5	70.2	58.6	53.1
MMMU-Pro _{overall} (Yue et al., 2024)	48.6 Chen et al. (2024d)	51.5	51.9	48.6	46.2	51.1	38.3	31.56
Math								
MathVista _{mini} (Lu et al., 2024)	72.3 Chen et al. (2024d)	67.7	63.8	72.3	70.5	74.8	68.2	62.3
MATH-Vision _{full} (Wang et al., 2024d)	32.2 Chen et al. (2024d)	-	30.4	32.2	25.9	38.1	25.1	21.2
MathVerse _{mini} (Zhang et al., 2024c)	51.7 Chen et al. (2024d)	-	50.2	51.7	-	57.6	49.2	47.6
General Visual Question Answering								
MegaBench (Chen et al., 2024b)	47.4 MiniMax et al. (2025)	52.1	54.2	45.6	46.8	51.3	36.8	28.9
MMBench-EN _{test} (Liu et al., 2023d)	88.3 Chen et al. (2024d)	82.6	83.4	88.3	86.9	88.6	83.5	79.1
MMBench-CN _{test} (Liu et al., 2023d)	88.5 Chen et al. (2024d)	83.5	82.1	88.5	86.7	87.9	83.4	78.1
MMBench-V1.1-EN _{test} (Liu et al., 2023d)	87.4 Chen et al. (2024d)	80.9	83.1	87.4	86.1	88.4	82.6	77.4
MMStar (Chen et al., 2024c)	69.5 Chen et al. (2024d)	65.1	64.7	69.5	68.3	70.8	63.9	55.9
MME _{sum} (Fu et al., 2023)	2494 Chen et al. (2024d)	1920	2328	2494	2483	2448	2347	2157
MuirBench (Wang et al., 2024a)	63.5 Chen et al. (2024d)	-	68.0	63.5	-	70.7	59.6	47.7
BLINK _{val} (Fu et al., 2024c)	63.8 Chen et al. (2024d)	-	68.0	63.8	-	64.4	56.4	47.6
CRPE _{relation} (Wang et al., 2024h)	78.8 Chen et al. (2024d)	-	76.6	78.8	-	79.2	76.4	73.6
HallBench _{avg} (Guan et al., 2023)	58.1 Wang et al. (2024f)	55.5	55.0	57.4	58.1	55.2	52.9	46.3
MTVQA (Tang et al., 2024)	31.9 Chen et al. (2024d)	25.7	27.8	31.9	30.9	31.7	29.2	24.8
RealWorldQA _{avg} (X. Al, 2024)	78.7 Chen et al. (2024d)	60.1	75.4	78.7	77.8	75.7	68.5	65.4
MME-RealWorld _{en} (Zhang et al., 2024f)	62.9 Chen et al. (2024d)	51.6	45.2	62.9	-	63.2	57.4	53.1
MMVet _{turbo} (Yu et al., 2024)	74.0 Wang et al. (2024f)	70.1	69.1	72.3	74.0	76.2	67.1	61.8
MM-MT-Bench (Agrawal et al., 2024)	7.4 Agrawal et al. (2024)	7.5	7.72	-	6.59	7.6	6.3	5.7

文本任务benchmark：

Table 4: Performance on pure text tasks of the 70B+ Instruct models and Qwen2.5-VL.

Datasets	Llama-3.1-70B	Llama-3.1-405B	Qwen2-72B	Qwen2.5-72B	Qwen2.5-VL-72B
General Tasks					
MMLU-Pro	66.4	73.3	64.4	71.1	71.2
MMLU-redux	83.0	86.2	81.6	86.8	85.9
LiveBench-0831	46.6	53.2	41.5	52.3	57.0
Mathematics & Science Tasks					
GPQA	46.7	51.1	42.4	49.0	49.0
MATH	68.0	73.8	69.0	83.1	83.0
GSM8K	95.1	96.8	93.2	95.8	95.3
Coding Tasks					
HumanEval	80.5	89.0	86.0	86.6	87.8
MultiPL-E	68.2	73.5	69.2	75.1	79.5
Alignment Tasks					
IFEval	83.6	86.0	77.6	84.1	86.3

在两个子任务上的表现也值得注意：OCR、Agent

Table 5: Performance of Qwen2.5-VL and other models on OCR, chart, and document understanding benchmarks.

Datasets	Claude-3.5 Sonnet	Gemini 1.5 Pro	GPT 4o	InternVL2.5 78B	Qwen2.5-VL 72B	Qwen2.5-VL 7B	Qwen2.5-VL 3B
OCR-related Parsing Tasks							
CC-OCR	62.5	73.0	66.9	64.7	79.8	77.8	74.5
OmniDocBench _{edit en/zh} ↓	0.330/0.381	0.230/ 0.281	0.265/0.435	0.275/0.324	0.226 /0.324	0.308/0.398	0.409/0.543
OCR-related Understanding Tasks							
AI2D _{w. M.}	81.2	88.4	84.6	89.1	88.7	83.9	81.6
TextVQA _{val}	76.5	78.8	77.4	83.4	83.5	84.9	79.3
DocVQA _{test}	95.2	93.1	91.1	95.1	96.4	95.7	93.9
InfoVQA _{test}	74.3	81.0	80.7	84.1	87.3	82.6	77.1
ChartQA _{test Avg.}	90.8	87.2	86.7	88.3	89.5	87.3	84.0
CharXiv _{RQ/DQ}	60.2 /84.3	43.3/72.0	47.1/84.5	42.4/82.3	49.7 /87.4	42.5/73.9	31.3/58.6
SEED-Bench-2-Plus	71.7	70.8	72.0	71.3	73.0	70.4	67.6
OCRBench	788	754	736	854	885	864	797
VCR _{En-Hard-EM}	41.7	28.1	73.2	-	79.8	80.5	37.5
OCR-related Comprehensive Tasks							
OCRBench_v2 _{en/zh}	45.2/39.6	51.9/43.1	46.5/32.2	49.8/52.1	61.5 /63.7	56.3/57.2	54.3/52.1

Table 9: Performance of Qwen2.5-VL and other models on GUI Agent benchmarks.

Benchmarks	GPT-4o	Gemini 2.0	Claude	Aguvis-72B	Qwen2-VL-72B	Qwen2.5-VL-72B
ScreenSpot	18.1	84.0	83.0	89.2	-	87.1
ScreenSpot Pro	-	-	17.1	23.6	1.6	43.6
Android Control High _{EM}	20.8	28.5	12.5	66.4	59.1	67.36
Android Control Low _{EM}	19.4	60.2	19.4	84.4	59.2	93.7
AndroidWorld _{SR}	34.5% (SoM)	26% (SoM)	27.9%	26.1%	6% (SoM)	35%
MobileMiniWob _{++SR}	61%	42% (SoM)	61% (SoM)	66%	50% (SoM)	68%
OSWorld	5.03	4.70	14.90	10.26	2.42	8.83